

Problem 1: Nobody's Happy (written) (15 points)

Is it possible for a stable matching to have no agent be matched to their first choice? If yes, give an example and explain why your example works. If no, prove why no such matching can exist

Answer:

Yes it is possible to have no agent matched with their first choice, let's go back to the xavier, yancey, and zeus example from class to observe this scenario

	1	2	3		1	2	3
Xavier	Shands	North	Vetrans	Shands	Yancy	Zeus	Xavier
Yancy	North	Vetrans	Shands	North	Zeus	Xavier	Yancy
Zeus	Vetrans	Shands	North	Vetrans	Xavier	Yancy	Zeus

In this set up the matching becomes (Xavier, North), (Yancey, Vetrans), and (Zeus, Shands) this example works through have each favorite for the hospital being the least favorite for the applicant and vice versa. With stable matching as stated in the lecture slides "N in a matching M, an unmatched pair a-h is unstable if applicant a and hospital h prefer each other to current assignments. N unstable pair a-h could each improve their choices. Stable matching: perfect matching with no unstable pairs. So in my example i took the edge cases (worst and best) for each agent and made the pair unstable thus forcing each agent to get it's second choice

Problem 2: Exploring Worst-Case Scenarios (written) (25 points)

In the stable matching algorithm described in class, students apply to companies offering internships in their preference order, and each company tentatively accepts the best student it has received so far, while rejecting less-preferred students. If there are n students and n companies, how many companies (as a function of n) can be forced to accept students who are their last choice? Describe a scenario that would cause this many companies to be matched with their last-choice students: what preference orderings would lead to this outcome, and how does the algorithm behave when given this input?

Answer:

If there are n students and n companies as a function of n there can be at most 1 company forced to accept their last choice.

	1	2	3		1	2	3
Company1	student2	student1	student3	student1	Company1	Company2	Company3
Company2	student1	student2	student3	student2	Company2	Company1	Company3
Company3	student1	student2	student3	student3	Company1	Company2	Company3

This leads to the matching (Company1, student1), (Company2, student2), and (Company3, student3) this is an example outcome of stable matching problem from slides.

I believe that at most one company obtains its worst choice and I'll describe this through contradiction. Essentially let's say there are two companies who are paired with their worst choices company1 and company 2. This entails that Company1 which was paired with its least preferred choice, Company1 must have looked at all other students and rejected each of them meaning every other company/student relationship had a better pair and thus the algorithm should end. This means that the algorithm stopped before company2 had the chance to pair with its worst pair, which contradicts that company1 and company2 were paired with their least preferred choice as once a least-preferred choice is found execution of the algorithm would have stopped because all matches should have been made by then, company2 being matched with its worst pair wouldn't allow for company1 to be matched with its worst pair and vice versa. At most 1 company gets its worst choice.

I wasn't really sure about how to go about this one or even if my logic is correct, these are the resources I used to help me come up with this answer

[1]man, "Prove that at most one man obtains his worst choice in stable matching algorithm," *Mathematics Stack Exchange*, Apr. 06, 2020.
<https://math.stackexchange.com/questions/3611845/prove-that-at-most-one-man-obtains-his-worst-choice-in-stable-matching-algorithm> (accessed Jan. 31, 2025).

[2]Wikipedia Contributors, "Gale–Shapley algorithm," *Wikipedia*, Feb. 11, 2020.
https://en.wikipedia.org/wiki/Gale%E2%80%93Shapley_algorithm

Problem 3: Implementing the Stable Matching Algorithm (programming) (60 points)

Hello here will be some documentation explaining the approach of my program including the algorithm, data structures used, and time complexity.

The first thing I worked on for this assignment is the parser for the inputs, as said in the discussion we were allowed to use the parse module. To use this module I first researched how to use it at: [3]"parse," *PyPI*, Jun. 11, 2024. <https://pypi.org/project/parse/> (accessed Feb. 01, 2025). This gives an explanation of the library and its uses. I described the steps to install the parse dependencies. The way that parse works is that it checks the first line for n giving the program the amount of companies and applicants. Then I had the program identify the locations of applicant preferences and company preferences in which it then used for loops to create a 2D list for applicant preferences and company preferences. This function is called in main as ParseFile and it returns the applicant preferences, the company preferences, and the amount of applicants/companies n.

To get the solution to the gale-shapley algorithm I implemented the stableMatching function which in general follows the general structure of the pseudocode that was found in the slides:

"Initialize each person and hospital to be free.

while (some hospital is free and hasn't been matched/assigned

to every applicant) {

Choose such a hospital h

a = 1st applicant on h's list to whom h has not been

matched

if (a is free)

assign h and a

else if (a prefers h to her/his current assignment h')

assign a and h, and h' has a slot free

else

a rejects h

}"

I created a ranking which is a 2D list that holds the priorities of an applicant for a company with lower rank having a higher preference for. Application matching and company matching are both just lists which provide the company/applicant currently match. I also have an index in the form of a list which tracks which company index each applicant will propose to next in their preference list. Finally I have the preference lists for both companies and applicants which are just a list of lists that contain order of preferences. At the beginning the company matches and applicant matches are set to be free, index is set to zero and then the while loop to go through

the algorithm starts (it stops when there are no free applicants). It first picks a free applicant, checks the company on the applicants list and if the company is free they match reducing the free applicant indicator by 1, if it's not free it checks which preference is better and swaps/frees the old match if necessary. Once all applicants have been matched it returns the stable matching solution as a list of (applicant, company)

I then created an output function which simply goes through the solution and writes it out to a file named output.txt with the specified formatting.

Main calls all of the functions and initializes the input file and output file as the txts in the zip file.

Here is an analysis on the time complexity of the Gale-Shapley algorithm within my program:

First building the ranking 2D list:

```
ranking = [[0]*n for _ in range(n)]

for company in range(n):

    for rank, applicant in enumerate(companyPrefs[company]):

        ranking[company][applicant] = rank
```

It loops over n companies and for each n companies we loop over all applicants thus we have an $O(n^2)$ loop here

Then for the while loop:

```
while freeApplicants > 0:

    applicant = None

    for i in range(n):

        if applicantMatch[i] == -1 and nextProposalIndex[i] < n:

            applicant = i

            break

    ...
```

There are n applicants and n companies and it is possible for every applicant to propose to every company ($n * n$) thus across the while loop you can have $O(n^2)$

Thus the total complexity of the Gale-Shapley algorithm should be $O(n^2) + O(n^2)$ which just becomes $O(n^2)$.